

AI BILLING DELEGATION STANDARD

ABDS v0.5

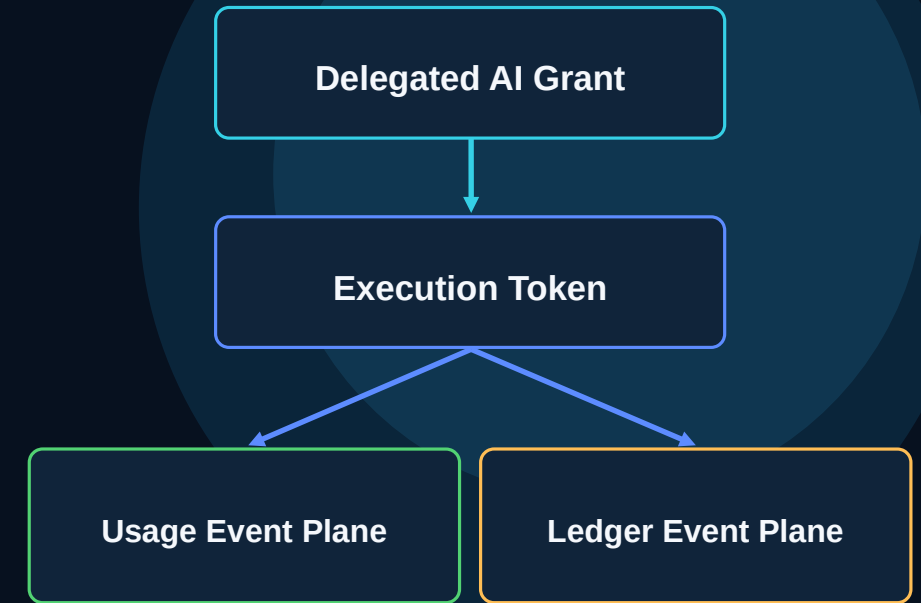
Provider-enforced funding, usage attribution,
and settlement for third-party AI applications

OAuth-aligned

Payer-neutral

Append-only events

Executive & Technical Brief



Why ABDS needed a v0.5 upgrade

The authorization core was necessary - but not sufficient for production cost control.

Legacy gap 1 - Invoice opacity

A Provider invoice cannot explain which user, workspace, feature, workflow, agent step, route, retry, or model attempt caused the cost.

Legacy gap 2 - Variable cost

Streaming, multimodal, batch and agentic operations do not have a reliable final cost when the request begins.

Legacy gap 3 - Hidden retries

One logical action may trigger several billable physical attempts through retries, fallback, routing or safety calls.

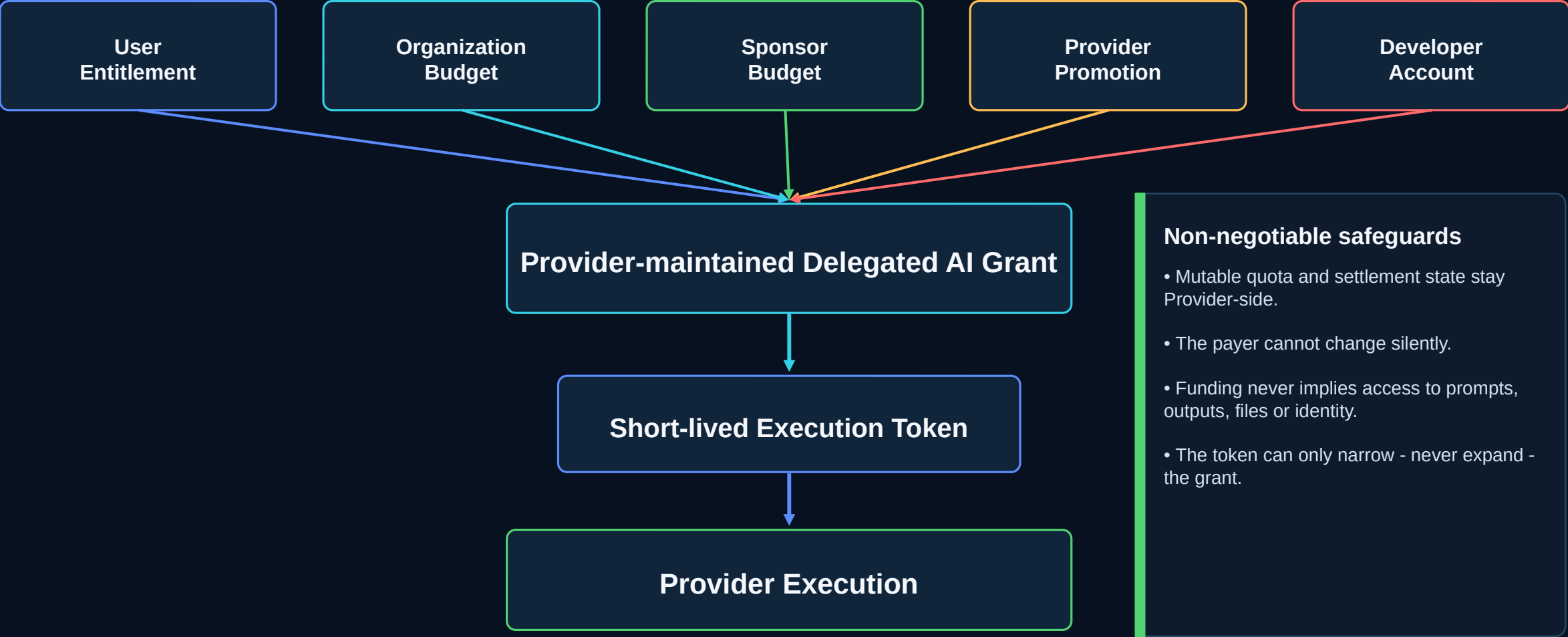
v0.5 adds the missing operational layer

- One immutable event per physical attempt
- Separate technical usage and economic ledger events
- Requested vs resolved model and route attribution
- Estimate - reserve - execute - settle - release
- Idempotency, reconciliation and compensating adjustments

From “who may spend?”
to
“who spent, why, where, and how was it settled?”

Payer-neutral authorization core

The funding party can differ from the person using the application.



The v0.5 two-plane architecture

Technical facts and economic mutations have different trust, privacy and correction requirements.

Provider Execution

```
graph TD; PE[Provider Execution] -- green arrow --> UEP[USAGE EVENT PLANE]; PE -- orange arrow --> ELP[ECONOMIC LEDGER PLANE];
```

USAGE EVENT PLANE

What technically happened

- logical request + physical attempt
- requested and resolved model
- Provider route, retry and outcome
- tokens, modalities, tools and latency
- optional workspace / workflow / agent refs

One immutable event per billable attempt.

ECONOMIC LEDGER PLANE

What happened economically

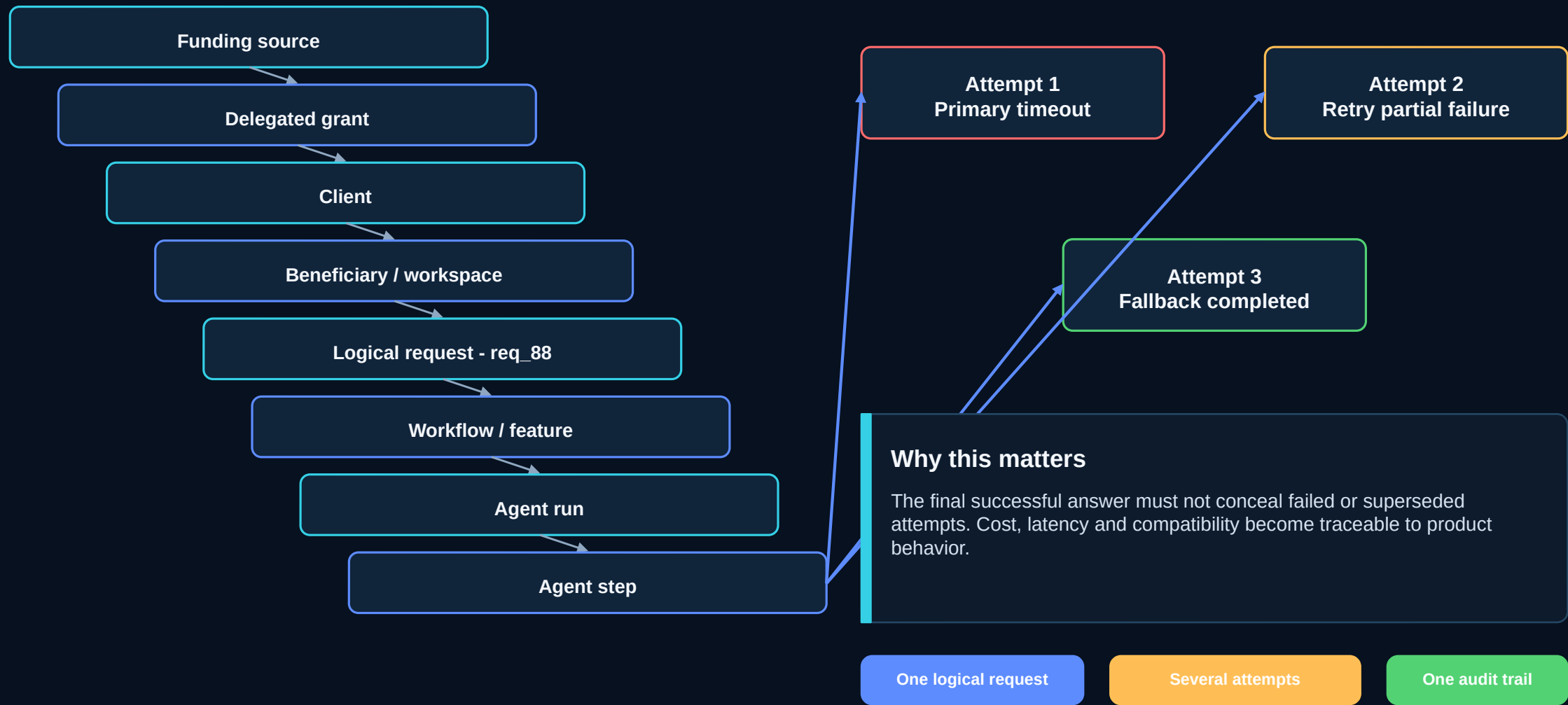
- reservation created
- settlement posted
- unused quantity released
- reservation expired
- execution denied
- compensating adjustment

Provider-authoritative and idempotent.

Client labels help explain product behavior, but cannot change the payer or billed quantity.

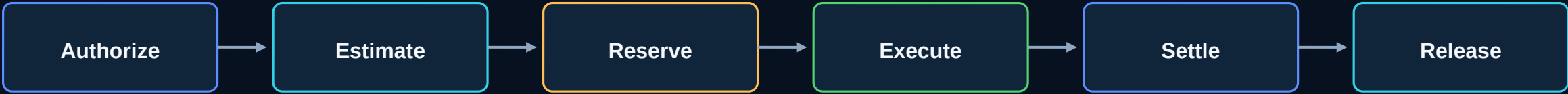
Attribution hierarchy and retry visibility

One user action can create several real executions - every billable attempt remains visible.



Reservation and settlement

Variable-cost execution is bounded before work begins and reconciled after usage is known.



Reservation

A temporary atomic hold bound to one grant, Client, logical request, unit type and funding bucket. It prevents concurrent overspend.

Settlement

The final idempotent debit uses Provider-measured usage and references every billable Usage Event.

Finalization

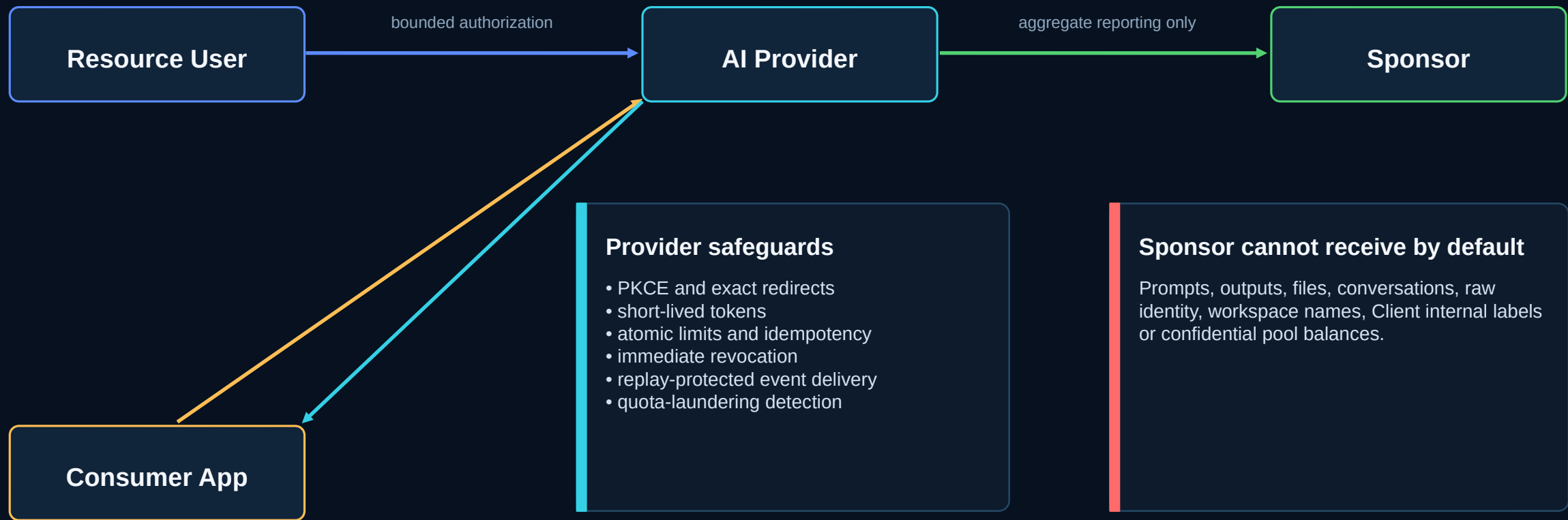
One reservation has one terminal finalization. Unused quantity is released inside settlement or through full release when no settlement occurs.

Accounting invariants

- one settlement ID accepted once
- settled + released <= reserved
- corrections use new adjustment events
- historical price snapshot retained

Privacy and security boundaries

Economic sponsorship, operational observability and user content are separate permissions.



Funding does not create a data-access right.

Current status and next proof point

v0.5 is a synchronized draft proposal - not yet a Provider-adopted standard.

COMPLETED IN v0.5

- ✓ canonical specification
- ✓ usage and ledger event profiles
- ✓ reservation and settlement
- ✓ JSON Schemas and examples
- ✓ automated validation
- ✓ discovery and implementation profiles
- ✓ expanded threat model
- ✓ synchronized diagrams and change control

NEXT - v0.6

- ABDS Studio simulator
- mock Authorization and Resource Servers
- grant and ledger service
- positive and negative test vectors
- retry, fallback and idempotency tests
- Sponsor privacy tests
- machine-readable conformance report

REVIEW NEEDED

- AI Provider platform engineers
- OAuth and identity experts
- billing and accounting architects
- security and abuse specialists
- privacy experts
- consumer AI developers
- Sponsor / nonprofit operators

Can an AI Provider enforce bounded third-party usage in a way that is auditable, payer-safe, privacy-preserving and interoperable?